

QMUI-Android 使用指南及替代方案

QMUI-Android 详细使用指南及替代方案

一、QMUI-Android 介绍

QMUI-Android 是由腾讯 QQ 音乐团队开源的 Android UI 组件库，旨在提升 UI 开发效率、统一设计风格，提供了丰富的组件（如导航栏、弹窗、列表、表单等）和工具类，支持自定义主题，兼容 Android 4.4+。

二、QMUI-Android 集成与基础使用

1. 集成方式（Gradle）

Step 1: 添加仓库

在项目根目录的 `build.gradle` 或 `settings.gradle` 中添加仓库：

Code block

```
1 // settings.gradle
2 dependencyResolutionManagement {
3     repositories {
4         mavenCentral()
5         maven { url 'https://jitpack.io' } // 若使用快照版
6     }
7 }
```

Step 2: 添加依赖

在模块的 `build.gradle` 中添加依赖：

Code block

```
1 dependencies {
2     // 核心库（最新版本参考GitHub）
3     implementation 'com.qmuiteam:qmui:2.1.0'
4     // 可选：QMUI 扩展库（如 RecyclerView 封装）
5     implementation 'com.qmuiteam:qmui-recyclerview:2.1.0'
6 }
```

Step 3: 配置主题

在 `res/values/styles.xml` 中继承 QMUI 主题：

Code block

```
1 <style name="AppTheme" parent="QMUI.Compat.NoActionBar">
2     <!-- 自定义主题属性 -->
3     <item name="qmui_primary_color">@color/colorPrimary</item>
4     <item name="qmui_primary_text_color">@color/colorPrimaryText</item>
5 </style>
```

2. 核心组件使用示例

(1) QMUI 工具栏 (QMUIActionBar)

替代原生 `ActionBar`，支持自定义左侧返回、右侧菜单：

Code block

```
1 <!-- 布局文件 -->
2 <com.qmuiteam.qmui.widget.QMUIWindowInsetLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent">
6
7     <com.qmuiteam.qmui.widget.QMUITopBar
8         android:id="@+id/topBar"
9         android:layout_width="match_parent"
10        android:layout_height="?attr/qmui_topbar_height"/>
11
12     <!-- 内容区域 -->
13 </com.qmuiteam.qmui.widget.QMUIWindowInsetLayout>
```

Code block

```
1 // Activity 中配置
2 QMUITopBar topBar = findViewById(R.id.topBar);
3 topBar.setTitle("QMUI Demo");
4 topBar.addLeftBackImageButton().setOnClickListener(v -> finish());
5 topBar.addRightTextButton("设置", R.id.topbar_right_setting)
6     .setOnClickListener(v -> { /* 点击事件 */ });
```

(2) QMUI 弹窗 (QMUIDialog)

替代原生 `AlertDialog`，支持自定义布局：

Code block

```
1 new QMUIDialog.MessageDialogBuilder(this)
```

```

2      .setTitle("提示")
3      .setMessage("这是QMUI弹窗示例")
4      .addAction("取消", (dialog, index) -> dialog.dismiss())
5      .addAction("确认", (dialog, index) -> {
6          // 确认逻辑
7          dialog.dismiss();
8      })
9      .show();

```

(3) QMUI 列表项 (QMUIListItemView)

快速实现带图标/箭头的列表项：

Code block

```

1  <com.qmuiteam.qmui.widget.grouplist.QMUICommonListItemView
2      android:layout_width="match_parent"
3      android:layout_height="?attr/qmui_list_item_height"
4      app:qmui_arrowDirection="right"
5      app:qmui_leftAccessoryType="image"
6      app:qmui_leftAccessoryDrawable="@drawable/ic_settings"
7      app:qmui_text="设置"/>

```

3. 自定义主题

QMUI 提供了丰富的主题属性，可在 `styles.xml` 中覆盖：

Code block

```

1  <style name="AppTheme" parent="QMUI.Compat.NoActionBar">
2      <!-- 主色调 -->
3      <item name="qmui_primary_color">@color/teal_500</item>
4      <!-- 工具栏高度 -->
5      <item name="qmui_topbar_height">56dp</item>
6      <!-- 圆角大小 -->
7      <item name="qmui_radius_large">12dp</item>
8  </style>

```

三、QMUI-Android 的优缺点

优点：

1. **组件丰富**：覆盖导航、弹窗、表单、列表等常用场景，减少重复开发；
2. **设计统一**：遵循 QQ 音乐的设计规范，风格简洁；
3. **兼容性好**：支持 Android 4.4+，适配全面屏、暗黑模式；

4. **工具类实用**：提供屏幕适配、状态栏管理、图片加载等工具。

缺点：

- 1. **体积较大**：集成后会增加 APK 体积（约 2-3MB）；
- 2. **更新较慢**：部分组件维护不及时，最新 Android 版本适配可能滞后；
- 3. **定制化成本**：部分组件样式强耦合，深度定制需修改源码。

四、QMUI-Android 替代方案

1. Jetpack Compose（官方推荐）

- **特点**：Android 官方声明式 UI 框架，组件轻量化、可组合性强，适配最新系统特性；
- **适用场景**：新项目或逐步迁移的项目，追求原生体验和灵活性；
- **示例**：

Code block

```
1  @Composable
2  fun QMUITopBar替代() {
3      TopAppBar(
4          title = { Text("Compose Demo") },
5          navigationIcon = { IconButton(onClick = {}) {
6              Icon(Icons.Filled.ArrowBack, null) } },
7          actions = { TextButton(onClick = {}) { Text("设置") } }
8      )
9  }
```

2. Material Design Components（MDC）

- **特点**：Google 官方 Material Design 实现库，组件全面、更新及时，支持 Material You 动态配色；
- **适用场景**：遵循 Material Design 规范的项目，追求设计一致性；
- **集成**：

Code block

```
1  implementation 'com.google.android.material:material:1.11.0'
```

- **示例**：

Code block

```
1  <com.google.android.material.appbar.MaterialToolbar
```

```
2     android:layout_width="match_parent"
3     android:layout_height="?attr/actionBarSize"
4     app:title="MDC Demo"
5     app:navigationIcon="@drawable/ic_back"
6     app:menu="@menu/menu_setting"/>
```

3. 鸿洋的 BaseRecyclerViewAdapterHelper (BRVAH)

- **特点：**专注于列表优化，替代 QMUI 的列表组件，支持多布局、上拉加载、下拉刷新；
- **适用场景：**列表需求复杂的项目；
- **集成：**

Code block

```
1     implementation 'com.github.CymChad:BaseRecyclerViewAdapterHelper:3.0.7'
```

4. SmartRefreshLayout

- **特点：**替代 QMUI 的下拉刷新组件，支持自定义 Header/Footer，适配 RecyclerView、ListView；
- **集成：**

Code block

```
1     implementation 'io.github.scwang90:refresh-layout-kernel:2.1.0'
```

5. 其他 UI 库

- **ButterKnife/Zelezny：**视图绑定工具（已被 ViewBinding 替代）；
- **ImmersionsBar：**状态栏/导航栏管理，替代 QMUI 的状态栏工具；
- **Coil/Glide：**图片加载库，替代 QMUI 的图片工具类。

五、使用建议

1. **新项目：**优先选择 **Jetpack Compose + MDC**，符合官方趋势，灵活性高；
2. **维护旧项目：**若需快速开发且设计风格匹配，可使用 QMUI，但需注意体积和兼容性；
3. **混合使用：**保留 QMUI 的核心组件（如工具栏、弹窗），结合 MDC 或 Compose 补充短板；
4. **轻量化需求：**直接使用系统原生组件 + 自定义工具类，减少第三方依赖。

总结

QMUI-Android 适合追求快速开发、设计风格统一的项目，但需权衡体积和定制化成本。对于新项目，建议优先采用官方技术栈（Compose/MDC），长期维护更有保障。若需兼容旧系统或复用现有组件，QMUI 仍是可行选择，但需结合替代方案优化关键模块。

（注：文档部分内容可能由 AI 生成）